

simp, simp sets and simp normal forms

Johannes Foltmann

1 Rewriting relations and their properties

To understand the design choices `simp`, `simp sets` and `simp`, we first need to take a look at formal rewriting relations and properties these rewriting relations can have:

Definition (Rewriting relation)

A **rewriting relation** on a set A is a binary relation $\rightarrow \subset A \times A$. We denote its reflexive transitive closure with $\rightarrow^*$.

Here, A can be interpreted as a set of formally defined terms, $s \rightarrow t$ as “ s can be rewritten into t in one step” and $s \rightarrow^* t$ can be read as “ s can be rewritten to t in finitely many steps”. Typical examples of rewriting relations include β - and η -reduction in (typed) λ -calculus, as well as `simp` in Lean.

1.1 Example: β - and η -reduction in λ -calculus

Definition (λ -terms)

In BNF-syntax, λ -terms are defined as follows:

$$e := x \mid \lambda x. e \mid e e$$

Essentially every formal type system, such as the type system at the core of Lean, is based on λ -terms. Therefore we can get an intuition on how to interpret λ -terms by looking at the corresponding terms in Lean.

	λ -term	In Lean
variables	x	<code>(x : α)</code>
λ -abstraction	$(\lambda x. t)$	<code>fun (x : α) => t</code>
application	$(f s)$	<code>f s</code>

Any λ -term can be interpreted as a function. λ -abstraction can be viewed as a way to define functions, by assigning a term t to the variable x . Function application can be seen as applying a function f to some term s .

Definition (One-step β -reduction)

One-step β -reduction is a rewriting relation on the set of λ -terms. Let x be a variable and t, s be λ -terms. Then

$$(\lambda x. t) s \rightarrow_{\beta,1} t (x := s)$$

Here, $(x := s)$ means, that we replace every occurrence of x with s .

β -reduction can be thought of as eliminating a λ -abstraction by plugging s in for x . Together with β -reduction, λ -calculus already forms a remarkably powerful formal system; in fact, it is Turing complete.

Definition (One-step η -reduction)

(One-step) η -reduction is the rewriting relation on λ -terms, such that for all terms f and variables x :

$$\lambda x. (f x) \rightarrow_{\eta,1} f, \text{ if } x \text{ is not free in } f$$

.

Intuitively, this rule ignores redundant abstractions. Often, it is combined with β -reduction into $\beta\eta$ -reduction.

1.2 Properties of rewriting systems

Rewriting relations can have a variety of different properties. For us, the following three are of special significance.

Definition (Weak normalization)

$t \in A$ is **in normal form** if there is no $s \in A$ such that $t \rightarrow s$.

$t \in A$ **has normal form** s , if $t \rightarrow^* s$ and s is in normal form.

$\rightarrow$ is **weakly normalizing (WN)**, if every $t \in A$ has a normal form.

Although in a weakly normalizing rewriting relation every term has a normal form, this does not automatically mean, that every chain of rewrites will yield a normal form. For this we need a slightly stronger definition:

Definition (Strong normalization)

$t \in A$ is **strongly normalizing** if there is no infinite sequence $t \rightarrow t_1 \rightarrow t_2 \rightarrow \dots$

$\rightarrow$ is **strongly normalizing (SN)**, if every $t \in A$ is strongly normalizing.

Note that $\rightarrow$ is strongly normalizing, if and only if every possible chain of rewrites eventually terminates in a normal form.

Definition (Confluence)

$\rightarrow$ is **confluent (CR)**, if $c \leftarrow^* a \rightarrow^* b$ implies that there exists a d , such that $c \rightarrow^* d \leftarrow^* b$.

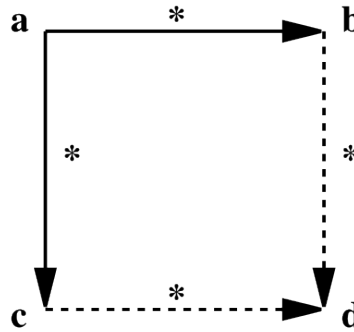


Figure 1: Confluence

Intuitively, confluence means that if rewriting diverges, we can always make it converge again. Usually in most rewriting relation this means, that it does not matter in which order we apply rewrites to subterms. The notation (CR) for confluence comes from the fact, that this property is also known as the Church-Rosser-property.

It is clear to see, that (SN) implies (WN). However (WN)/(SN) and (CR) are independent of each other. It is simple to find minimal rewriting systems, in which one property holds and the other does not. Together (WN) and (CR) imply that normal forms are unique, because if they were not, (CR) would imply the existence of a common successor of two different normal forms, which contradicts them being normal forms. However, (SN) alone does *not* imply unique normal forms.

In untyped λ -calculus, β -reduction is not normalizing. To see that, define $\omega := \lambda x.x x$. Then:

$$\omega \omega = (\lambda x.x x) \omega \rightarrow_{\beta,1} \omega \omega \rightarrow_{\beta,1} \dots$$

In contrast, in simply typed λ -calculus β -reduction is strongly normalizing, and by the Church-Rosser theorem, β -reduction is confluent.

2 Basics of simp

`simp` is a conditional rewriting system whose goal is to simplify terms into shorter, more readable expressions. It replaces terms and subterms using sets of lemmas, called `simp` sets, traversing terms down to the leaves and rewriting innermost subterms first. Beyond rewriting with lemmas, `simp` also performs reduction operations such as β -, η -, ζ -, and ι -reduction, as well as projections.

`Simp`-lemmas are of the form $A = B$ or $A \leftrightarrow B$, and `simp` always replaces A with B . A lemma is added to the standard `simp` set with the `[@simp]` attribute; definitions can be tagged with `[@simp]` in the same way. Internally, `simp`-lemmas are structured in a discrimination tree.

2.1 Basic Example

We can apply `simp` either as a tactic in proof mode or as a command, which will return the terms simplified form.

```
/-Enable simp diagnostics-/
set_option
  trace.Meta.Tactic.simp.rewrite true

variable (G : Type) [Group G] (a b c : G)

example : a * a-1 * 1 * b
  = b * c * c-1 := by simp

#simp a * a-1 * 1 * b
#simp b * c * c-1
```

2.2 simp with hypotheses

We can add hypotheses (locally) to the `simp` set with `[]`:

```
example (h : b = 1) : a * b = a := by simp [h]
```

Using `simp at ...` simplifies statements in hypotheses.

```
example (h0 : d = a * a-1 * 1 * b)
(h1 : e = b * c * c-1) : d = e := by
  simp at h0 h1; simp only [h0, h1]

example (h0 : d = a * a-1 * 1 * b)
(h1 : e = b * c * c-1) : d = e := by
  simp [*] at * --stronger: simp_all
```

2.3 Adding/removing lemmas

Lemmas can also be added to or removed from the `simp` set. Removing lemmas is especially useful, when a `simp`-lemma causes a loop or obstructs a more useful lemma from being used. In the following example, `not_and` rewrites the term into a useless form before `not_and_or` can simplify it, and is therefore removed from the `simp` set.

```
variable (A B C : Prop)

example (h : ¬A ∨ ¬B ↔ C) :
  ¬(A ∧ B) ↔ C := by
  simp [←h]; sorry

example (h : ¬A ∨ ¬B ↔ C) :
```

```

¬ (A ∧ B) ↔ C := by
  simp [not_and_or, ←h]; sorry

example (h : ¬A ∨ ¬B ↔ C):
  ¬ (A ∧ B) ↔ C := by
    simp [-not_and, not_and_or, ←h]

```

2.4 simp?

simp? suggests a minimal simp set:

```

example : a * a-1 * 1 * b
  = b * c * c-1 := by simp?

example : a * a-1 * 1 * b
  = b * c * c-1 := by
  simp only [mul_inv_cancel, mul_one,
    one_mul, mul_inv_cancel_right]

```

2.5 simp! and an interesting example

simp! automatically unfolds definitions:

```

#simp Nat.factorial 3

example : 6 = Nat.factorial 3 :=
  by simp!

```

2.6 dsimp

dsimp only uses lemmas that hold by definition (i.e. are proved by rfl):

```

example : 6 = Nat.factorial 3 :=
  by dsimp! only

def times_2 : ℤ →+ ℤ where
  toFun := fun n => 2 * n
  map_zero' := by dsimp
  map_add' := by {intros a b; ring}

```

2.7 simpa

This is a “finishing” tactic, so it should only be used at the end of a proof. simpa applies assumption after simp:

```

example (h : d = b):
  d = b * c * c-1 := by simpa

```

simpa [...] using h simplifies h and closes the goal with h:

```

example (h : d = a * a-1 * 1 * b):
  d = b * c * c-1 := by simpa using h

```

3 Simp sets and simp normal forms

In general, one cannot guarantee confluence and normalization for simp sets; the aim is instead to get as close as possible by designing the simp set to optimize for these properties.

3.1 A bad simp lemma

```
lemma Nat_add_zero' (n : ℕ) : n = n + 0 := by simp
```

Why would tagging this `@[simp]` be a bad idea? First, it is not normalizing: this lemma replaces `n` infinitely often. Second, it is redundant, since the lemma is already proved by `simp`. Third, it is too specific, as the lemma holds for `AddZeroClass` in general.

3.2 Designing simp sets

When designing simp sets, one should aim for confluence and normalization, avoiding simp lemmas that cause loops or that would be redundant. Libraries should not add simp lemmas that do not mention a constant they define, since doing so might change the behavior of earlier, unrelated simp calls.

3.3 Simp normal forms

Definition (Simp normal form)

The simp normal form (SNF) of an expression e is the result of applying `simp` with the default simp set.

This corresponds to the normal form with respect to `simp` viewed as a rewriting relation; note that equivalent terms might not have the same SNF. The SNF of an expression can be found with `#simp`:

```
#simp ¬(A ∧ B)
#simp ¬A ∨ ¬B
```

3.4 SNF in `@[simp]` lemmas

In a simp-lemma, the left-hand side should be in simp normal form, since otherwise the lemma might not be called; the `simpNF` linter checks for this. The right-hand side should be in (or close to) simp normal form as well, which ensures that `simp` terminates.

4 More details

4.1 Terminal vs. non-terminal_simps

Definition (Terminal simp call)

A `simp` call is terminal if it closes the goal.

A non-terminal `dsimp` is fine, since the result barely changes as the library grows. A non-terminal `simp`, on the other hand, is fragile: a new simp-lemma added later can change what `simp` does at that point in the proof. The usual solution is to turn a non-terminal `simp` into `simp only [...]`, found using `simp?`.

4.2 Priorities of simp lemmas

The default priority of a simp lemma is 1000; when multiple simp-lemmas are applicable, simp goes from highest to lowest priority. Named priorities are `low = 100`, `mid = 500`, `default = 1000`, and `high = 10000`, and priorities are written as `@[simp high]`, `@[simp 1200]`, or `@[simp low + 1]`.

4.3 An example from Mathlib

```
@[simp high] theorem add_eq_left' {a b : Nat} :  
  a + b = a ↔ b = 0 := by omega  
@[simp high] theorem add_eq_right' {a b : Nat} :  
  a + b = b ↔ a = 0 := by omega  
@[simp high] theorem left_eq_add' {a b : Nat} :  
  a = a + b ↔ b = 0 := by omega  
@[simp high] theorem right_eq_add' {a b : Nat} :  
  b = a + b ↔ a = 0 := by omega
```

These lemmas are very specific and very strong, so they should have priority over more general lemmas.

4.4 Another use case of priorities

Sometimes a simp-lemma declared later proves an earlier simp-lemma. Normally, one would `un-@[simp]` the earlier lemma or tag it with `@[simp, noint simpNF]`. However, if the earlier lemma is required by a large part of the library, or if the more general lemma makes simp slow, it can make sense to instead prioritize the earlier lemma with `@[simp high]`.

5 Conclusion

`simp` is a (conditional) rewriting system that replaces terms and subterms with lemmas from the simp set. The goal in designing a simp set is for `simp` to be as confluent and normalizing as possible, and the left-hand side of simp lemmas should always be in simp normal form.

```
set_option trace.Meta.Tactic.simp.rewrite true
```

References

- Lean Language Reference: *The Simplifier*:
<https://lean-lang.org/doc/reference/latest/The-Simplifier/>
- Lean Community Documentation: *Simp*:
<https://leanprover-community.github.io/extras/simp.html>